Medical Document Assistant

Classification + RAG Chatbot for Medicine Leaflets (English / Arabic)
Final Project Plan - AI Engineering Course (110 Hours)

1. Project Overview

A web application where a user uploads or pastes a medicine leaflet, prescription, or health article. The system classifies the document, extracts key facts (dosage, warnings, side effects), and lets the user ask questions through a chatbot that answers only from the uploaded text.

Goal: build an end-to-end AI system from data to deployed model, covering every module of the course.

Disclaimer: the app is informational only and is not medical advice. When the answer is not in the document, the chatbot must say so instead of guessing.

2. What We Are Going To Build

- **Document classifier:** predicts the drug category (for example antibiotic, painkiller, anti-inflammatory) from the leaflet text.
- **Information extractor:** pulls out dosage, warnings, contraindications, and side effects.
- **RAG chatbot:** answers user questions using retrieved passages from the uploaded document.
- **Web app:** a Streamlit interface backed by a FastAPI service, deployed online.
- **Model comparison report:** baseline ML vs. deep learning vs. fine-tuned BERT.

3. Work Plan by Course Module

Phase 1 - Python for AI (15h) | Key output: Data + EDA pipeline

- Pull drug label data from openFDA (and DailyMed bulk files if needed).
- Clean and normalize the text: remove HTML, fix encoding, split into sections.
- Build class labels from the pharmacologic class field.
- Run EDA: class balance, text length, most common terms, missing values.
- Save a clean dataset (CSV/Parquet) and a reusable cleaning script.

Phase 2 - Machine Learning (25h) | Key output: Baseline classifier

- Convert text to features with TF-IDF.
- Train Logistic Regression, Linear SVM, and Random Forest models.
- Use train/validation/test splits and cross-validation.
- Evaluate with accuracy, precision, recall, F1, and a confusion matrix.
- Keep the best model as the benchmark to beat.

Phase 3 - Deep Learning (35h) | Key output: PyTorch text model

- Build a tokenizer/vocabulary and a PyTorch Dataset and DataLoader.
- Train an LSTM or 1D-CNN text classifier.
- Tune learning rate, batch size, and dropout; add early stopping.
- Compare results with the Phase 2 baseline.
- Optional: a small autoencoder or embedding experiment to visualize document clusters.

Phase 4 - NLP and LLMs (25h) | Key output: Fine-tuned BERT + RAG chatbot

- Fine-tune a BERT model with HuggingFace (English BERT, or a multilingual/Arabic model such as AraBERT for Arabic).
- Split documents into chunks and create embeddings with a sentence-transformer model.
- Store embeddings in a vector database (FAISS or Chroma).
- Retrieve top matching chunks for each question and pass them to an LLM to generate a grounded answer.
- Add a fallback response: 'I cannot find that in the document.'
- Test answers against MedQuAD-style question/answer pairs.

Phase 5 - Capstone (10h) | Key output: Deployed application

- Wrap the models in a FastAPI backend (endpoints for classify, extract, and ask).
- Build a Streamlit frontend with file upload, results view, and chat.
- Containerize with Docker and deploy (Hugging Face Spaces, Streamlit Cloud, or a cloud VM).
- Write the README, architecture diagram, and final presentation (5-7 minutes).

4. Tools and Technologies

Area	Tools	Used For
Language	Python 3	All development
Data handling	NumPy, Pandas, requests	Data download, cleaning, EDA
Visualization	Matplotlib, Seaborn	EDA charts, confusion matrices
Classical ML	Scikit-learn (TF-IDF, Logistic Regression, SVM, Random Forest)	Baseline classifier and evaluation
Deep learning	PyTorch	LSTM / CNN text classifier
NLP / LLMs	HuggingFace Transformers, Datasets, sentence-transformers	BERT fine-tuning, embeddings
Vector search	FAISS or Chroma	Retrieval for the RAG chatbot
LLM generation	An LLM API or an open model from HuggingFace	Grounded answer generation
Backend	FastAPI, Uvicorn	REST API serving the models
Frontend	Streamlit	Upload, results, and chat interface
Deployment	Docker, Hugging Face Spaces / Streamlit Cloud	Public demo
Dev workflow	Google Colab or Kaggle (GPU), Git and GitHub, Jupyter	Training, version control, notebooks

5. Data Sources

Source	Content	Use
openFDA Drug Label API	Full drug label text by section	Main document corpus and class labels
DailyMed (NIH/NLM)	Bulk drug label downloads	Larger offline corpus
RxNorm / RxClass, WHO ATC	Drug names and class mappings	Label scheme for the classifier
Kaggle: UCI Drug Review	Patient reviews by condition	Extra text for NLP experiments
MedQuAD / PubMedQA	Medical question-answer pairs	Testing the chatbot
Arabic sources	Egyptian Drug Authority info, MedlinePlus Arabic, Hugging Face Arabic medical datasets, or translated samples (marked as synthetic)	Arabic extension (after English works)

Check each source's license and terms before use, and keep the data for educational purposes.

6. Evaluation

- **Classifier:** accuracy, macro F1, and confusion matrix on a held-out test set, compared across all three model families.
- **Chatbot:** check whether answers are supported by the retrieved text, and how often it correctly says it cannot find an answer.
- **App:** response time and a short manual test with sample leaflets.

7. Deliverables

- GitHub repository with clean code and a README
- Cleaned dataset and EDA notebook
- Trained models and a model comparison table
- Live demo link (FastAPI + Streamlit)
- Architecture diagram
- Final presentation (5-7 minutes)

8. Time Allocation (Based on Course Hours)

Phase	Course Module	Hours
1	Data collection, cleaning, EDA	15h
2	Baseline ML classifier	25h
3	Deep learning model	35h
4	BERT fine-tuning + RAG chatbot	25h
5	API, UI, deployment, presentation	10h
Total		110h

9. Risks and Mitigation

Risk	Mitigation
Arabic data is limited	Build English first; add Arabic as an extension, using clearly labeled translated data if needed
Imbalanced classes	Merge rare classes, use class weights, report macro F1
Chatbot gives wrong medical information	Answer only from retrieved text, show the source passage, add a clear disclaimer
Limited GPU time	Use Colab/Kaggle GPUs and a smaller BERT variant
Deployment problems	Deploy early with a simple version, then improve